

ENTERPRISE DATA PLATFORM — ARCHITECTURE, OPERATIONS & REFERENCE V1

The layers, the decisions that are hard to reverse, and what actually breaks in production
Ingestion · storage & table formats · transformation · orchestration · quality · governance · serving · cost · depth marked junior · staff

1 · WHAT A DATA PLATFORM IS FOR — AND THE FOUR DECISIONS THAT ARE HARD TO REVERSE

The definition
The system that moves data from **where it is produced to where decisions are made** — reliably, on time, with known quality, and under governance. Everything on this sheet is in service of those four needs, and a platform that delivers data fast but untrustworthily has failed as completely as one that delivers nothing.

Why it is layered at all. Each layer exists so that change on one side does not propagate to the other. A source system changes its schema; the raw layer absorbs it. An analyst needs a new metric; the curated layer serves it without re-ingesting anything. The layers are a **decoupling mechanism**, not a **pipeline diagram** — and treating them as stages to pass through rather than boundaries to defend is the most common architectural mistake.

What good looks like

Trusted	Consumers know the quality and can ask where a number came from. This is harder and more valuable than any pipeline
Timely	Freshness meets a stated requirement, and someone is told when it does not
Discoverable	People can find data without seeing a where
Governed	Access is controlled, sensitive data is protected, and both are provable
Affordable	Cost is attributed and someone owns it

The four decisions that are genuinely hard to reverse

- Table format and catalog.** This determines which engines can read your data, what transactional guarantees you have, and how locked in you are. See panel 5 — it is the single most consequential technical choice on the sheet
- Partitioning and layout strategy.** Rewriting the physical layout of petabytes is expensive and slow. Getting it wrong shows up as cost and latency on every query, forever
- The operating model.** Centralised, federated or mesh — this is an organisational decision that determines who can move quickly and who becomes a bottleneck. Changing it means reorganising people
- What the raw layer keeps.** If you did not read it immediately, you cannot reprocess it. Data not captured on day one is gone, and no later decision recovers it

Everything else is more changeable than it looks. Orchestration, transformation tools, BI layers and query engines can all be swapped with effort. The four above are structural — so they deserve most of the design time, and they usually get least.

The failure mode worth naming **first**: a **too list** is not an architecture. Most platform diagrams are inventories of products. What determines whether a platform works is the **contracts between the layers** — what each guarantees the next, who owns each boundary, and what happens when one breaks. Two organisations with identical tools routinely get opposite outcomes.

The questions that expose a weak design

- “A source changes a column type tomorrow — what happens?”** If the answer is a broken dashboard discovered by a user, there is no contract
- “A pipeline produced wrong data last Tuesday. How do you fix it?”** The inevitable raw data, idempotent transforms and a backfill mechanism at once
- “Who owns this table?”** If nobody can answer, governance is aspirational
- “What does this dashboard cost to run?”** Cost that is not attributed to cost is not managed
- “A customer invokes deletion rights. What do you do?”** Deletion across immutable storage, backups and downstream copies is genuinely hard — see panel 11

A note on nomenclature terminology. Bronze, silver and gold is Databricks' naming for raw, cleaned and business-ready layers. The idea is older and vendor-neutral — the same three-layer pattern appears everywhere under different names. Use whichever your organisation knows, and be aware the terms carry a vendor association.

2 · THE LAYERS — WHAT EACH ONE GUARANTEES THE NEXT

Layer	Its job, and its contract
Sources	Applications, operational databases, event streams, files, third-party APIs. You do not control them — which is why the next layer must absorb their change
Ingestion	Move data in, without transforming it. Contract: deliver everything, exactly once or idempotently, and record what arrived. Failure here is invisible later, because you cannot miss data you never saw
Raw / Bronze	Store exactly as received , immutably, partitioned by arrival. Contract: reprocessing is always possible. This layer is the platform's insurance policy and the reason a bad transform is recoverable
Processing	Clean, deduplicate, conform, join, enrich. Contract: deterministic and idempotent — running it twice produces the same result, which is what makes backfills safe
Curated / Silver	Conformed, quality-checked, modelled entities. Contract: this data is trustworthy and its schema is stable
Serving / Gold	Business-ready aggregates and marts, shaped for consumption. Contract: fast, correct, and matching agreed definitions
Consumption	BI, notebooks, applications, ML features, APIs. Different consumers need genuinely different shapes

JUNIOR what each layer holds **STAFF** the contract it owes downstream, and how it fails

The rule that makes the raw layer worth its cost: never transform on the way in. Every cleaning step applied during ingestion is a decision you cannot revisit without re-extracting from a source that may no longer hold the data. Land it as it came, **including the malformed records**, and clean it in the next layer where the logic is versioned and re-runable.

Schema-on-read to schema-on-write

The raw layer is deliberately **permissive** — it accepts whatever arrives so nothing is lost. Each subsequent layer becomes **stricter**, until the serving layer has a firm, contract-backed schema. Rigour increases as you move right, and that gradient is the design.

The common anti-pattern: a **curated layer robbery** trunks. When teams bypass it and build directly from raw, you get parallel definitions of the same metric and no single source of truth. The cause is almost never that you move right, and that gradient is the design. **Fix the turnaround time, not the access controls.**

A connection on ingestion too long

Apache Spark **should not appear in a new design. It was retired to the Apache Atlas in 2021**, with its last release in 2017. For relational sources today: **change data capture** with Debezium for continuous replication, managed connectors such as Airbyte or Fivetran, Kafka Connect, or a straightforward Spark JDBC extraction.

Prefer change data capture over scheduled full extracts wherever the source supports it. It is lower load on the operational database, gives near-real-time freshness, and — importantly — **captures deletes**, which a query-based incremental extract silently misses.

3 · BATCH VS STREAMING

the decision, honestly

Start here: most **stated real-time requirements are not real-time**. Asked precisely, “we need this in real time” usually means **within fifteen minutes** — which a scheduled batch job delivers at a fraction of the cost and complexity. **Ask what decision changes if the data is an hour old.** If nothing does, batch is the answer.

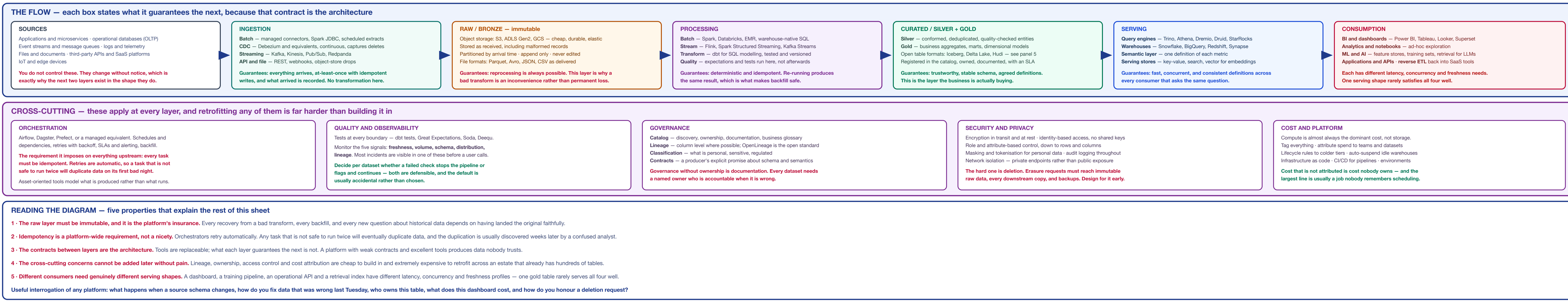
	Batch	Streaming
Latency	Minutes to hours	Sub-second to seconds
Cost	Lower — compute runs and stops	Higher — always on
Complexity	Modest	Substantially higher
Reprocessing	Re-run the job	Replay the stream — needs planning
Debugging	Inspected inputs and outputs	Harder — state is in flight

When streaming genuinely earns its cost

- Fraud and abuse detection** — the decision must happen before the transaction completes
 - Operational alerting** on equipment or systems
 - Live personalisation** where the session ends before a batch would run
 - Event-driven integration** between systems, where the stream is the interface rather than an analytics input
- Concepts that **only appear in streaming**
- Event time versus processing time** — when it happened, versus when you saw it. Almost all correctness bugs live in this gap
 - Watermarks** — how long to wait for late data before closing a window. **Too short loses data, too long delays every result**
 - Exactly-once** — achievable end to end, and it requires idempotent or transactional sinks. “At least once plus idempotent writes” is usually simpler and sufficient
 - State management** — windows and joins hold data that must be checkpointed and restored
- The pragmatic middle is **micro-batch** — running every few minutes with streaming tooling but batch semantics. It covers the large majority of “near real-time” needs with far less operational burden than true event-at-a-time processing.

On Lambda and Kappa: Lambda runs batch and streaming paths in parallel and reconciles them — two implementations of the same logic, which drift. Kappa treats the stream as the single source and replays it for reprocessing. Modern table formats make a single pipeline serving both patterns practical, which is why Lambda is now largely a legacy shape rather than a target design.

4 · REFERENCE ARCHITECTURE — THE FLOW, AND THE CONCERNS THAT CUT ACROSS IT



5 · FILE FORMATS, TABLE FORMATS AND THE CATALOG — THREE DIFFERENT THINGS

These are routinely tested against each other and operate at different layers. Parquet and iceberg are not alternatives — an iceberg table is made of Parquet files. Conflating them is the most common conceptual error in lakehouse diagrams, and it makes the actual decision invisible.

CATALOG	Which tables exist, who may read them, where metadata lives
TABLE FORMAT	Iceberg, Delta, Hudi, ACID, time travel, evolution
FILE FORMAT	Parquet, ORC, Avro, how bytes are laid out on disk
OBJECT STORE	S3, AZLS Gen2, GCS

File formats — how bytes are stored

Parquet	Columnar. The default choice for analytics. Reads only the columns needed, compresses well because a column holds similar values, and carries statistics that let engines skip whole row groups
ORC	Also columnar, with similar properties. Common in Hive-lineage estates
Avro	Row-oriented with a strong schema. Good for streaming and for writing whole records; poor for analytical scans of a few columns
JSON / CSV	Interchange only. Uncompressed, untyped, slow — acceptable in the raw layer because that is what arrived, never in the curated layer

Table formats — what turns files into a table

- A directory of Parquet files is not a table. A table format adds a metadata layer that provides what a database gives you for free:
- ACID transactions** — a writer's changes appear atomically, so readers never see a half-written update
 - Time travel** — query the table as it was at a past point, which makes debugging and auditing tractable
 - Schema evolution** — add, rename and reorder columns safely
 - Partition evolution (iceberg)** — change the partitioning without rewriting history, which directly addresses one of the four hard-to-reverse decisions
 - Concurrent writers with conflict detection**
 - Efficient row-level updates and deletes** — including the ones private key requires

The three options, iceberg — vendor-neutral, the broadest engine support, and the strongest partition-evolution story. **Delta Lake** — mature and excellent, historically strongest within the Databricks ecosystem. **Hudi** — designed around upserts and incremental pulls, strong for CDC-heavy ingestion.

STAFF The important recent development: the format argument is settling into interoperability. The iceberg REST catalog specification has become the common interface, implemented by Snowflake, AWS Glue, Google BigLake, Dremio and Databricks Unity Catalog. Apache Polaris, an open REST catalog implementation, graduated to a top-level project in February 2026. Databricks added native iceberg support in 2025. Iceberg v2 brings deletion vectors, row lineage and a variant type. The practical consequence is that the choice is less locking than it was two years ago — but the catalog is now where lock-in actually lives.

So choose the catalog as deliberately as the format. It holds table metadata, enforces access control, and determines which engines can participate. Ask: does it implement the open REST specification, can multiple engines write as well as read, and what happens to your metadata if you leave?

6 · SCHEMA CHANGE & DATA CONTRACTS

the top cause of breakage

The single most common way a data platform breaks: an upstream team changes a schema without knowing anyone depended on it. No notice and no process forces them on their part — they simply had no visibility of consumers. Every durable fix for this is organisational before it is technical.

Which changes are safe

Change	Effect
Add a nullable column	Safe. Existing readers ignore it
Add a required column	Breaks readers that write to the table
Rename a column	Breaking. Add the new name, migrate consumers, then remove the old one
Drop a column	Breaking for anyone using it. Deprecate first, with a date
Widen a type	Usually safe
Narrow or change a type	Breaking, and often silently — values are truncated or coerced rather than rejected
Change semantics	The worst case. The schema is unchanged, so nothing detects it, and every downstream number is quietly wrong

That last row deserves emphasis. A field whose meaning changes — currency shifted, a status code repurposed, a timestamp moved from local to UTC — passes every schema check and every null test. It is caught only by **distribution monitoring** or by a human noticing. This is the strongest argument for owning semantics explicitly rather than relying on structural validation.

Data contracts — the modern answer

- A **versioned, explicit agreement** published by the producer: the schema, the semantics of each field, freshness and quality guarantees, the owner, and the deprecation policy.
- Checked in CI** on the producing side, so a breaking change fails the producer's build rather than the consumer's discovery
 - Versioned**, with breaking changes requiring a new version and a migration window
 - It moves the cost to where the decision is made** — which is the entire point, and why it works where documentation does not
- Start with a schema registry for streams and enforce compatibility rules — it is the cheapest version of this idea and delivers most of the benefit. Full contracts across every dataset are a programme; a registry on your highest-value streams is a sprint.

7 · TRANSFORMATION AND MODELLING — WHERE ANALYTICS ENGINEERING LIVES

ETL became ETLT, and the reason matters
Traditional ETL transformed data afterwards, because storage and compute were expensive and scarce. With cheap object storage and elastic compute, the order inverted: **load raw, then transform in place**. The benefit is that the raw data is still there when the transformation turns out to be wrong — which it will.

dbt is the dominant tool for this layer and deserves naming explicitly. It brings software engineering practice to the SQL transformation: **version control, modular models with dependency resolution, tests, documentation and lineage generated from the code itself**. Whatever you use, those are the properties to insist on.

Modelling — the classical patterns still apply

- Star schema**: Facts surrounded by dimensions. **Sell the right answer** for most BI serving — it is comprehensible, performs well, and analysts already know it
- Wide tables**: One denormalised table per subject. Simple and fast on columnar storage; duplicated logic and grows unwisely
- Data vault**: Hubs, links and satellites. Excellent auditability and history, considerable complexity — justified in heavily regulated settings, rarely elsewhere
- Slowly changing dimensions**: How you handle attributes that change. **Type 1** overwrites and loses history. **Type 2** adds a row with validity dates and preserves it. **Decide deliberately** — the wrong choice is discovered when someone asks a historical question you cannot answer

MID ETL and dbt **STAFF** dimensional modelling still applies, and idempotency is the constraint

Idempotency is the constraint that shapes every transformation. Orchestration retries. Backfills re-run. Late data arrives and a window must be re-computed. A transform that appends rather than replacing its target partition will **duplicate data on the first retry** — and the duplication is typically found weeks later by an analyst wondering why a total is too high.

How to make it idempotent

- Overwrite by partition** rather than appending. Re-running a day replaces that day's output entirely
- Merge on a key**, so a re-run updates rather than inserting again
- Derive the output solely from the input** — no dependence on wall-clock time, no accumulating state on the side
- Make the run window an explicit parameter**, so re-running a past period is the ordinary path rather than a special one

Incremental processing and backfill

- Full refresh** — simple, correct, and eventually too slow or too expensive
- Incremental** — process only what is new. Requires a reliable high-water mark, and must handle **late-arriving data** that lands behind the mark
- Backfill must be a first-class capability**, not a manual script. You will need it for a bug fix, a new column over history, and a definition change — plan for all three
- Bound backfills** launched. A naive backfill of two years of daily partitions launched at once will saturate the cluster and can cost more than the original build

Test transformations like code, because they are code. Uniqueness and not null on keys, referential integrity between models, accepted values on enumerations, and row-count deltas against the previous run. These catch most breakage before a consumer does, which is the entire objective.

8 · TABLE MAINTENANCE

the operational reality nobody plans for

The small files problem — the most common lakehouse complaint. Frequent writes, especially streaming or per-partition micro-batches, produce thousands of tiny files. Every query then opens thousands of objects, and **metadata and request overhead dominates the actual reading**. Queries slow steadily with no change to data volume or query text, and object-store request charges climb alongside.

The maintenance jobs every table format needs

Job	What it does
Compaction	Rewrites many small files into fewer large ones. The single highest-value maintenance task , and the one most often left unscheduled
Clustering and sorting	Physically co-locates related rows so queries skip more data. Sometimes called Z-ordering or liquid clustering depending on the platform
Snapshot expiry	Removes old table versions. Directly trades time-travel depth against storage cost — pick the retention deliberately rather than inheriting a default
Orphan file removal	Deletes files left by failed writes, which no metadata references and nothing else will ever clean up
Statistics refresh	Keeps query planning accurate as data grows

Schedule these jobs on day one. They are unglamorous; they are usually nobody's explicit responsibility, and a lakehouse without them degrades steadily until someone investigates a slow dashboard six months later and finds a table with two million files.

- Partitioning — and over-partitioning**
- Partition on what queries filter on**, most often a date. A partition that no query uses is pure overhead
- Aim for substantial partitions** — a useful working target is hundreds of megabytes to a few gigabytes each
- Over-partitioning is as damaging as under-partitioning**. Partitioning by hour and by customer produces millions of tiny partitions, and planning alone becomes the bottleneck
- High-cardinality partition columns are almost always wrong** — cluster or sort on their instead

Iceberg's partition evolution and hidden partitioning reduce the cost of getting this wrong — the partitioning can change without rewriting history, and queries need not know the partition scheme to benefit from it. It is one of the more practically valuable reasons to choose it.

9 · ORCHESTRATION

more than a scheduler

The orchestrator owns dependencies, retries, alerting, backfill and the record of what ran. A cron job runs things on time; an orchestrator tells you what it depends on what and what it does when something fails.

- What it must provide
- Dependency resolution** — a task runs when its inputs are genuinely ready, not at a time someone guessed would be late enough
- Retries with backoff**, and a clear distinction between transient and permanent failure
- Backfill** as a first-class operation over an arbitrary date range
- SLA** — alert when a dataset is late, not merely when a task errors. Late and silent is the more dangerous failure
- Observability** — run history, duration trends, and where time is actually spent
- Parameterisation**, so the same pipeline runs for any window

Task-oriented versus asset-oriented. Traditional orchestrators model **tasks** — units of work in a graph. Newer ones model **assets** — the datasets produced, with the graph derived from what depends on what. The asset view maps better onto how people actually reason about a platform: they care that a table is fresh, not that a job succeeded.

The anti-pattern: one enormous DAG. A single graph containing every pipeline becomes impossible to reason about, and one failure blocks unrelated work. **Split by domain and ownership**, and connect across boundaries with sensors or asset dependencies rather than shared tasks.

Practices worth insisting on

- Every task idempotent** — a hard requirement, given automatic retries
- Pipelines in version control**, reviewed and deployed through CI
- No credentials in DAG code** — use the secret backbone
- Alerts route to an owner**, not a shared inbox nobody reads
- Task duration trended** — a job that has quietly grown from four minutes to fifty is a warning before it is a warning before it is an incident

10 · DATA QUALITY & OBSERVABILITY

what to measure

Data observability rests on five signals. Nearly every incident is visible in one of them before a user notices — which is the difference between finding your own problems and being told about them.

Signal	Catches
Freshness	Did it arrive on time? The highest-value single check — most incidents start here
Volume	Row counts against expectation. A partial load looks like success and delivers half the data
Schema	Structure changed — new, missing or retyped columns
Distribution	Values changed shape — null rate, cardinality, ranges, category mix. The only automated defence against a change in meaning
Lineage	What is upstream and, more importantly, who is affected when this breaks

Tests, and where they belong

- At ingestion** — did we get what we expected, in the volume we expected?
- After each transformation** — uniqueness, not-null, referential integrity, accepted values
- Before publishing** — business rules, totals reconcile, no negative quantities, dimensions resolve
- Continuously on serving tables** — freshness and volume, alerting to an owner

The decision that must be explicit: does a failed check stop the flag and continue? Stopping protects consumers from wrong data and makes the dataset late. Continuing keeps it fresh and may publish something innocent. **Both are legitimate and they suit different datasets** — a financial report should stop, an engagement dashboard probably should not. What is never acceptable is nobody having decided.

Publish an SLA per dataset — freshness, expected volume, quality checks, and the owner. It converts “the data looks wrong” from an argument into a measurement, and it gives the platform team a defensible definition of done.

11 · GOVERNANCE, SECURITY & PRIVACY

including the hard part

The layers of control

Catalog	What exists, what it means, who owns it. Discovery without asking a person is the measure of whether it works
Lineage	Column level where attributes, answers “where did this number come from” and “what breaks if I change this”. Open lineage is the open standard
Classification	Tag what is personal, sensitive or regulated — automatically where possible, since manual classification decays
Access control	Role and attribute-based, with row and column filtering. Grant roles where possible , since more individuals
Masking	Dynamic masking by role; tokenisation where the value must be joinable but not readable
Audit	Who accessed what, and when. Required in most regulated environments and useful in all

The genuinely hard problem: deletion. A privacy request to erase an individual must reach **immutable raw files, every derived table, aggregates, exports, backups and any downstream SaaS copy**. Most platforms discover they cannot do this only when the first request arrives.

Designing for deletion in advance

- Keep personal data in few, known places** and reference it by key elsewhere — this alone makes erasure tractable
 - Use a table format with efficient row-level deletes**, so erasure does not mean rewriting a partition
 - Crypto-shredding** — encrypt per subject and destroy the key. **Often the only practical answer for backups**
 - Track lineage to downstream copies**, including reverse-ETL destinations that left the platform
 - Set retention deliberately**. Data you no longer hold is data you need not delete, secure or explain
- Governance succeeds or fails on ownership. A catalog with no named owner per dataset is documentation. **Someone accountable when data is wrong** is the mechanism; the tooling only records it.

Controls worth putting in place

- Tag everything by team**. Domain and environment — attribution must exist before optimisation is possible
- Show teams their spend**. Visibility alone reduces cost measurably, before any technical change
- Lifecycle policies** moving cold data to cheaper tiers automatically
- Query limits and timeouts**, so one runaway query cannot produce a memorable bill
- Right-size on measurement**, not on the initial guess
- Budgets and anomaly alerts**, so a change is noticed within days rather than at month end

The framing that works with engineers' cost is a design constraint, not a procurement problem. A partitioning choice, a refresh interval and a file size are all cost decisions made by engineers — and they are far more effective than negotiating the contract.

12 · SERVING AND CONSUMPTION — MATCHING THE SHAPE TO THE CONSUMER

Four consumers, four different requirements

Consumer	Needs
BI and dashboards	Low latency at high concurrency , stable definitions, modest data volumes per query. Pre-aggregation and caching pay off here more than anywhere
Analytics and notebooks	Flexible access to detail, tolerant of slower queries, unpredictable shapes. A query engine over the lake fits well
ML and AI	Large historical extracts for training, low-latency lookups for inference , and — critically — a one feature computed identically in both
Applications and APIs	Millisecond-point lookups at a premium. An analytical store is the wrong tool — publish into a serving database instead

The mistake this table exists to prevent: **pointing a customer-facing application at the warehouse**. It works during the pilot and fails at production concurrency, generally with a cost surprise obscuring the latency one. Analytical engines are built for scans, not for thousands of concurrent point reads.

Query engine or warehouse?

- Query engines over the lake** — Trino, Athena, Dremio, Starburst. Data stays in open formats, several engines can read it, and you keep control of storage
- Cloud warehouses** — Snowflake, BigQuery, Redshift. Excellent performance and concurrency with far less to operate, historically at the cost of data living in a proprietary format
- The distinction is narrowing** — warehouses increasingly read and write open table formats directly, which makes a hybrid estate genuinely practical
- Real-time analytical stores** — Druid, Pinot, ClickHouse, StarRocks — for sub-second aggregation over fresh data at high concurrency

The semantic layer is the fix for a specific, universal problem: the same metric computed differently in four dashboards. Defining metrics once, in versioned code, and serving every tool from that definition is what stops meetings that debate whose revenue number is correct. It is one of the highest-leverage additions to a platform that already works.

Serving AI and ML workflows

- Feature stores** exist to solve **training-serving skew** — the same feature computed one way in a training pipeline and another way at inference, producing a model that performs worse in production than in evaluation. Define once, serve both
- Vector stores and vector-capable databases** hold embeddings for semantic retrieval — increasingly a standard platform component rather than a specialist one
- Retrieval pipelines are data pipelines** — the same requirements for freshness, lineage, quality and access control apply. Documents fed to a language model inherit their permissions from somewhere, and that somewhere should be your governance layer
- Reproducibility** — knowing exactly which data trained a model. Time travel makes this straightforward and its absence makes it nearly impossible

Reverse ETL pushes curated data back into operational tools — the CRM, the support platform, the marketing system — so insight reaches people where they work. **Treat those destinations as governed consumers**: they are copies that leave your perimeter, and they must appear in lineage and in your deletion process.

13 · COST

where the money actually goes

Compute, deletion, and storage. Object storage is inexpensive enough that teams optimise it out of habit while the actual bill is warehouse credits, cluster hours and query scanning. **Look at compute first** — it is almost always where the surprise is.